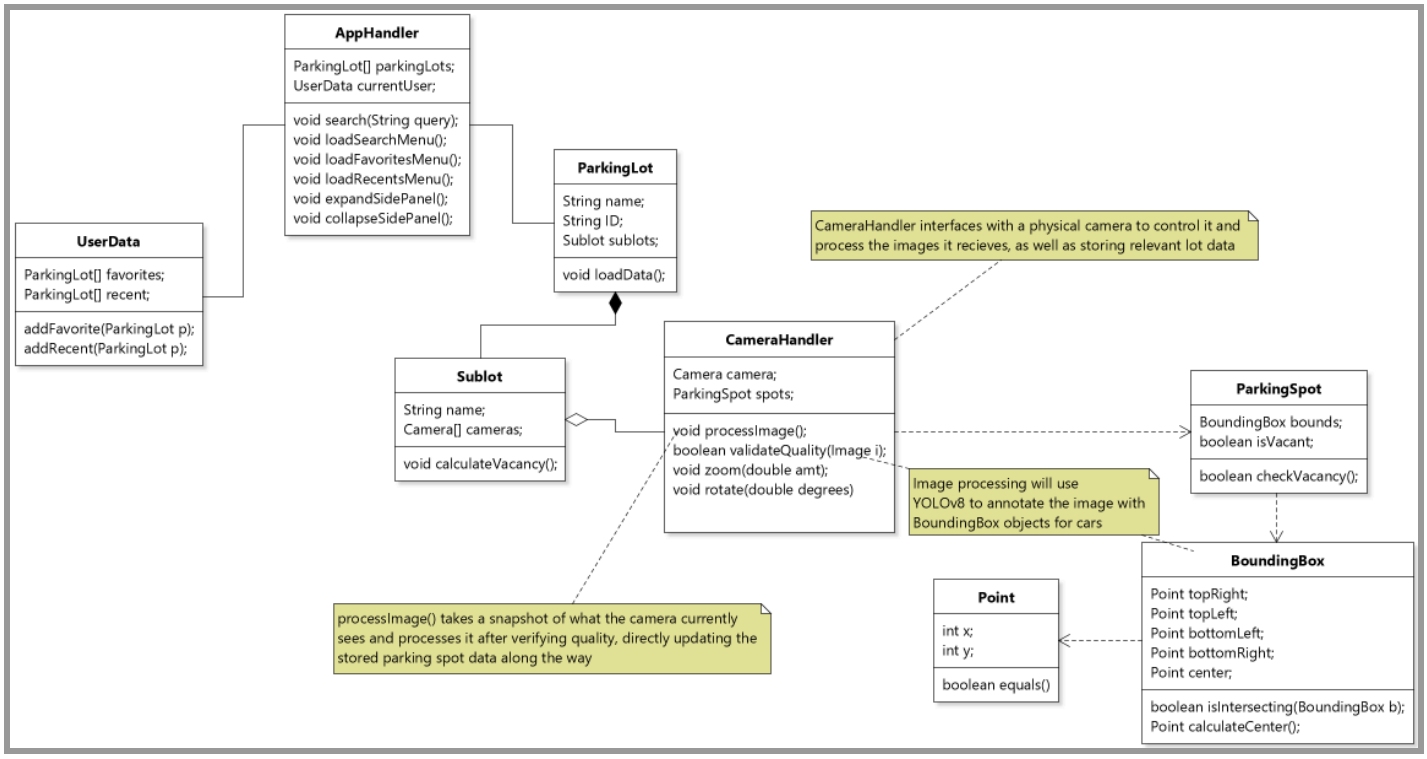
PARKING Design Documentation

Class Hierarchies:



An important distinction to make here is the difference between a “parking lot” and a “sublot”. A parking lot is the entirety of what we are examining as a whole. A sublot is some subsection of that parking lot. Any parking lot can be broken down into one or more sublots.

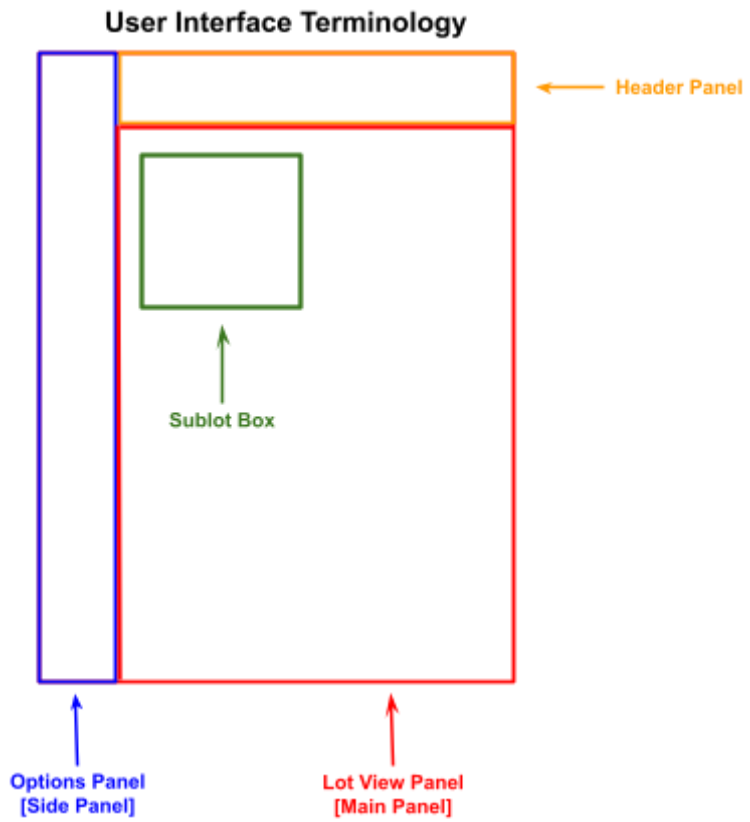
For example, think about how a very large parking lot may be divided into Lot A, Lot B, and so forth. It’s all one lot that has simply been broken down into logical sections so it’s easier to parse.

Each sublot will have a number of camera managers. There will be one camera manager per camera, and the number of cameras depends on how many cameras are needed to fully cover the sublot. These camera managers are responsible for keeping track of a certain set of parking spots.

When the database is queried for information, the data will flow bottom-up; CameraManagers will process information and pass it up to their sublot, and the Sublot aggregate all of the information it receives and pass it up to the ParkingLot who passes it to the app itself.

The database will update its information based on a timer. After some specified amount of time has elapsed, the database will update its information by querying all the parking lots to update their information. Then, the application will be signaled to query that new data

User Interface Design:



Lot View Panel: Displays information about the parking lot. Information is contained in “Sublot Boxes”. There are as many sublot boxes as there are sublots as defined by the parking lot. One parking lot is loaded, and the sublots are just how that lot is presented. The lot view panel is vertically scrollable.

Sublot Box: Contains the name of the sublot, as well as the number of vacant spots versus the number of total spots. The border of the box changes color based on the vacancy status:

Green = High vacancy
Yellow = Medium vacancy
Red = Low vacancy

The level of vacancy is determined by the number of spots available as a percentage value, so it's relative to the total number of spots.

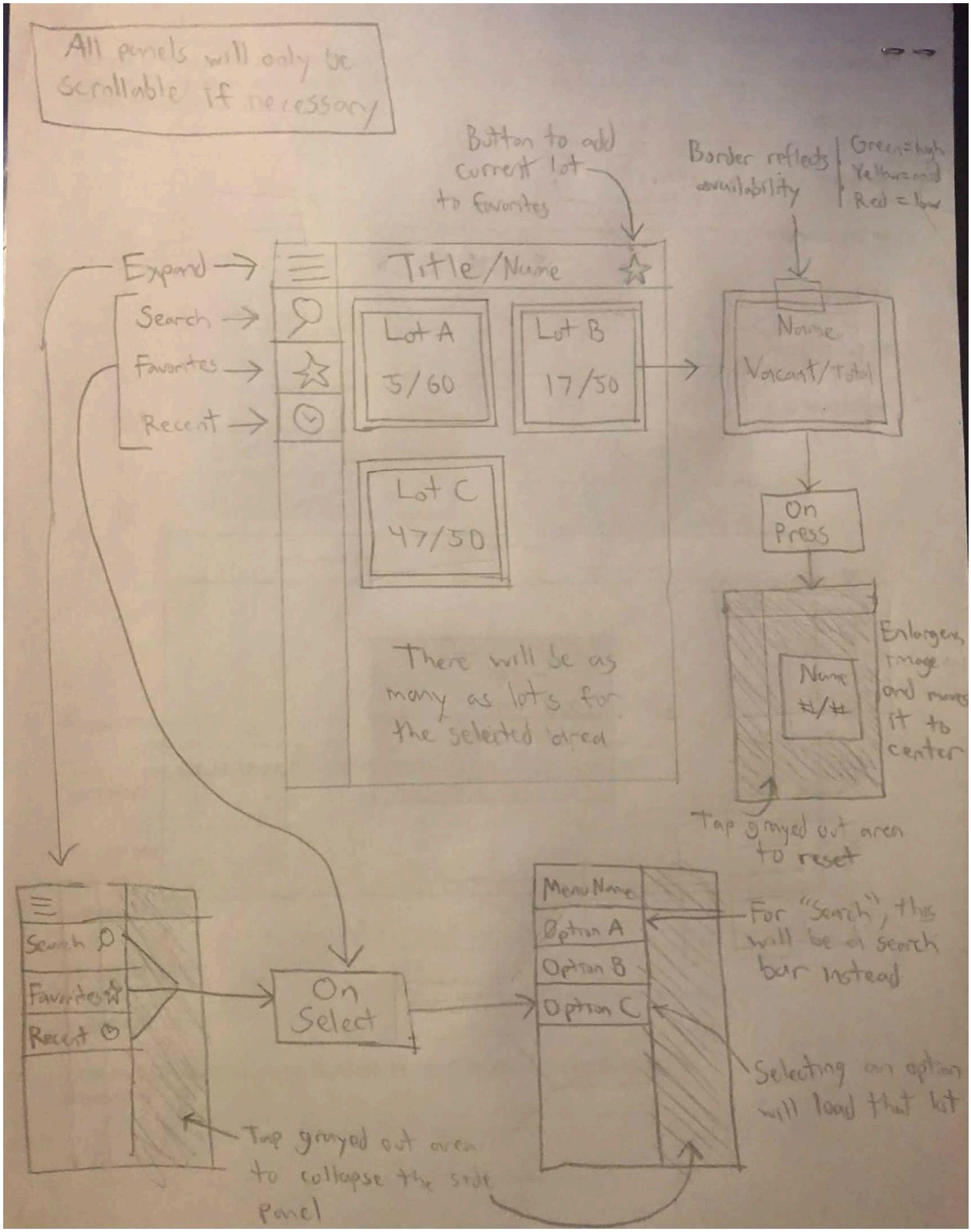
Header Panel: Displays any header information for the parking lot. Contains the display name for the lot as well as a favorites button.

Options Panel: Contains any viewing options that the user is able to select. Selecting any of the options will expand the options panel. Tapping anywhere outside of the options panel in the expanded state will collapse it to normal size.

This panel is vertically scrollable.

> **Expand:** Expands the options panel rightward and gives text descriptions for each symbol.
>
> **Search:** Brings up a search menu and prompts the user to type in a search bar. As the user types, any applicable parking lots based on the query will fill into the search menu.
>
> **Favorites:** Removes options from the options panel and fills it with all of the favorites stored in the user's in-app data. Favorites are added/removed by interacting with the "favorite" button in the header panel
>
> **Recents:** Removes options from the options panel and fills it with all of the recent selected lots stored in the user's in-app data. Recent lots data is automatically updated whenever the user selects a lot. The lot is added to a list and, if the list exceeds a certain size (say, 6), the oldest entry is removed.

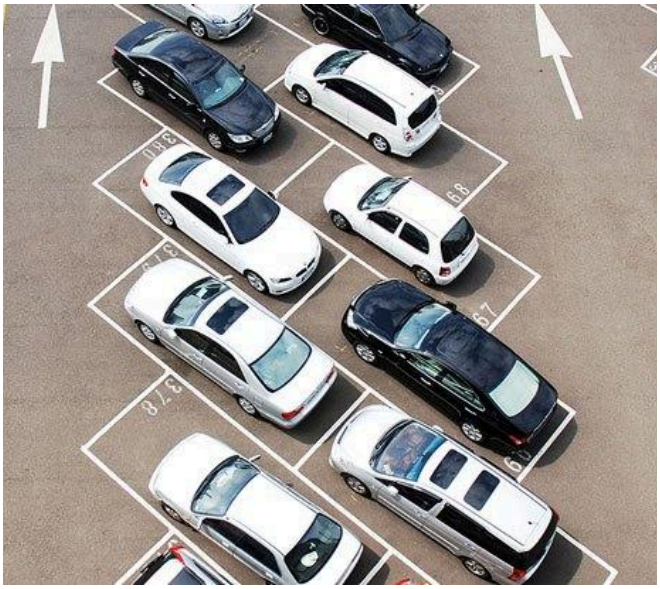
Selecting a parking lot from Search, Favorites, and Recent will load that lot. Loading a lot means that the header panel and lot view panel are filled with the data pertaining to the parking lot and the options panel is automatically collapsed.



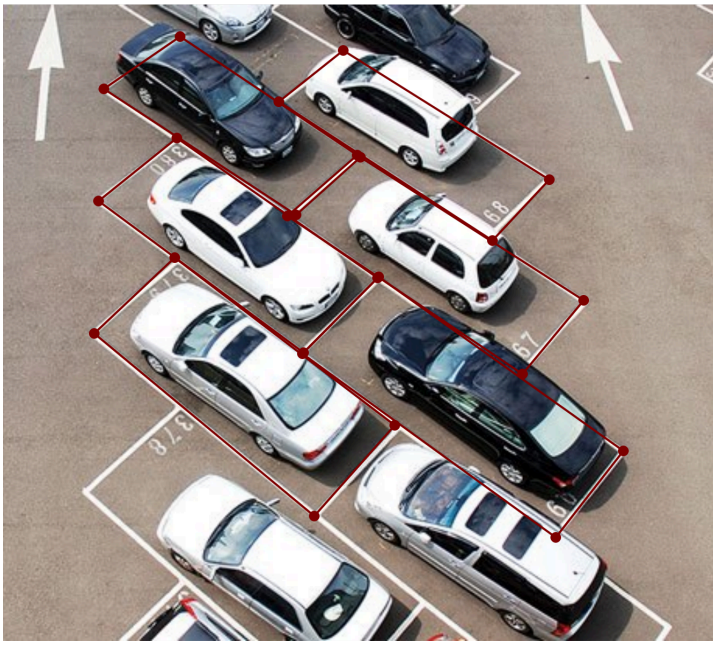
Shown above is a more detailed view of what the user interface would look like. This example depicts a parking lot that has been loaded and is stored in the database as three different sublots.

Image Processing:

We will use YOLOv8, a machine-learning algorithm, to detect cars in images using a pre-trained model. By cross-examining the output with a reference image that is annotated to show where the parking spots are we can determine which spots are occupied and which spots aren't.



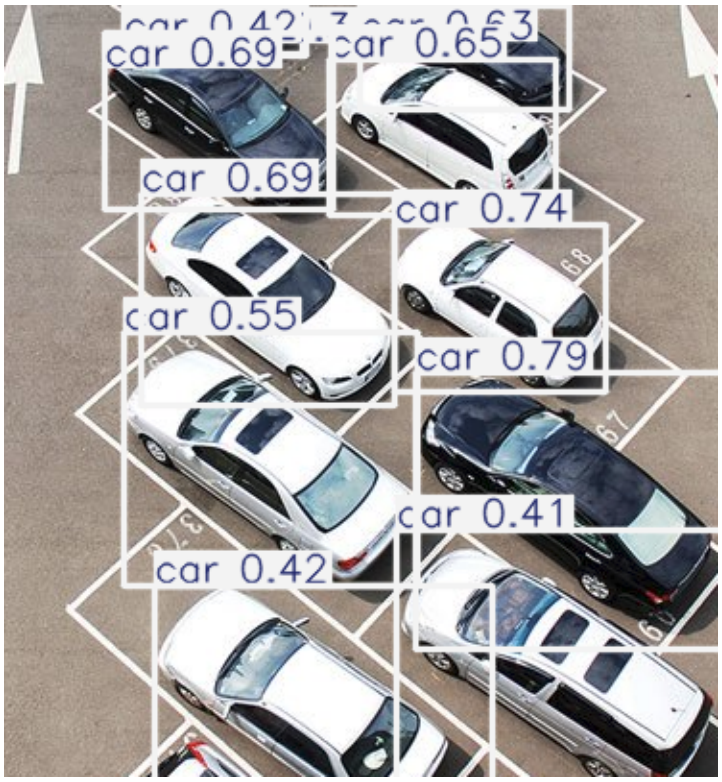
Here is an example image of a parking lot, from the relative angle we would be working with.



Here is how that example image would be annotated.

Each parking spot that is fully visible would be marked with a quadrilateral polygon, as shown in red.

These annotations are then saved as pixel coordinates which are stored in the database as a BoundingBox for the ParkingSpot object.



Here is the output when that original image is given to YOLOv8 to process.

Next to each object is a box showing where YOLOv8 determined the object is. It also says what the object was identified as and with what confidence that deduction was made.

Similar to the annotations, these boxes are stored as pixel coordinates. By comparing pixel coordinates of the detected cars against our reference, we can determine any overlaps. An overlap means that spot is occupied.

Since these calculations are based on pixel coordinates, it is crucial that the camera position, zoom, and angle used by the reference image are the exact same as the image being processed.

Only one reference image must be processed per camera angle, as that one reference is completely valid until the angle changes.